КНИГА 4

Практическое применение ИИ

Инструменты, интеграции, агенты и RAG — от повседневного использования до разработки

Уровень: Начинающий — Продвинутый
Библиотека ИИ · 5 книг · все уровни и категории

1. Карта инструментов: что для чего использовать

Начинающий

Инструментов на базе ИИ сейчас сотни, но большинство укладывается в несколько категорий по задаче, а не по бренду. Разумнее выбирать инструмент по типу задачи, а не по хайпу вокруг конкретного названия.

Задача	Тип инструмента	Пример категории
Общие вопросы, тексты, код	Чат-ассистент общего назначения	Claude, ChatGPT, Gemini
Поиск по своим документам	RAG-система / корпоративный поиск	Notion AI, внутренние базы знаний
Автономные многошаговые задачи	Агент с доступом к инструментам	Claude Code, исследовательские агенты
Генерация изображений/видео	Мультимодальные генеративные модели	Midjourney, Sora-подобные системы
Встраивание ИИ в своё приложение	API / SDK	Anthropic API, OpenAI API

2. Чат-интерфейс vs API: в чём разница

Начинающий

Чат-интерфейс (сайт или приложение) — для разовых диалогов человека с моделью. API (Application Programming Interface) — способ, которым ваша собственная программа отправляет запросы модели и получает ответ автоматически, без участия человека на каждом шаге.

```
# Пример вызова API (псевдокод, структура одинакова у большинства провайдеров)
response = client.messages.create(
    model="модель-id",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "Объясни квадратичную формулу простыми словами"}
    ]
)
print(response.content)
```

Примечание. API обычно тарифицируется по числу «токенов» (кусочков текста, см. Книгу 3) на входе и выходе, а не по количеству запросов — длинный контекст стоит дороже.

3. Что такое агент на практике

Средний

В отличие от обычного чат-ответа, агент выполняет цикл: получает задачу -> решает, какой инструмент вызвать (поиск, код, файл, другой API) -> выполняет вызов -> смотрит на результат -> решает, что делать дальше, повторяя цикл, пока задача не будет выполнена или не потребуется подтверждение человека.

```
Цикл агента (упрощённо):
while задача не завершена:
    план = модель.решить_следующий_шаг(история, доступные_инструменты)
    если план.нужен_инструмент:
```

```
        результат = вызвать_инструмент(план.инструмент, план.аргументы)
        история.добавить(результат)
    иначе:
        вернуть план.финальный_ответ
```

Ключевой практический вопрос при проектировании агента — не «может ли модель это сделать», а «какие действия безопасно разрешить без подтверждения человека, а какие требуют паузы» (например: чтение файла — обычно безопасно; удаление или платёж — обычно требует подтверждения).

4. RAG: как модель отвечает по вашим документам

Средний

RAG (Retrieval-Augmented Generation, «генерация с подкреплением поиском») решает проблему: модель не знает содержимое ваших внутренних документов, потому что они не входили в её обучение. Вместо дообучения модели на ваших данных (дорого и негибко) в запрос перед генерацией ответа подмешивается релевантный фрагмент текста, найденный поиском.

```
Шаги типичного RAG-пайплайна:
1. Документы разбиваются на небольшие фрагменты (чанки)
2. Каждый фрагмент превращается в embedding-вектор (см. Книгу 3) и сохраняется в векторной базе
3. На вопрос пользователя находятся N наиболее похожих по смыслу фрагментов
4. Эти фрагменты вставляются в промпт как контекст перед вопросом
5. Модель отвечает, опираясь на предоставленный контекст, а не только на свои знания
```

Примечание. RAG снижает количество «выдуманных» фактов, но не устраняет их полностью — модель всё ещё может неверно интерпретировать найденный фрагмент. Качество RAG-системы сильно зависит от качества разбиения на чанки и точности поиска, а не только от самой языковой модели.

5. Автоматизация рутинных задач

Начинающий

Самые надёжные практические применения ИИ сегодня — не «заменить человека полностью», а снять рутину там, где цена ошибки невысока и результат легко проверить:

- Черновики писем, описаний, постов — человек правит перед отправкой
- Суммаризация длинных документов и встреч
- Первичная сортировка обращений/заявок по категориям
- Генерация шаблонного кода, тестов, документации по образцу
- Перевод и адаптация текста под разные аудитории

6. Выбор между готовым инструментом и своей разработкой

Средний

Готовый инструмент (чат-интерфейс, no-code платформа) быстрее запустить, но даёт меньше контроля над поведением и данными. Своя разработка через API даёт полный контроль над промптами, доступом к инструментам и обработкой ошибок, но требует инженерных ресурсов на поддержку. Практическое правило: для разового или редкого использования — готовый инструмент; для встраивания в

продукт или процесс, который должен работать стабильно и масштабироваться, — API.

7. Безопасная интеграция: ключи, права доступа, лимиты

Продвинутый

При встраивании ИИ в реальный продукт или рабочий процесс важны те же базовые правила безопасности, что и для любого внешнего API:

- Никогда не хранить API-ключи в коде, который попадает в публичный репозиторий
- Давать агенту доступ только к тем инструментам и данным, которые реально нужны для задачи
- Ставить лимиты (бюджет, число запросов) — агент может зациклиться и сгенерировать неожиданно много запросов
- Логировать действия агента, особенно необратимые (запись в базу, отправка сообщений, платежи)
- Фильтровать пользовательский ввод перед передачей в инструменты с побочными эффектами (SQL, shell-команды)

8. Частые ошибки при внедрении ИИ в работу

Средний

- Доверие результату без проверки в задачах с высокой ценой ошибки (юридические, медицинские, финансовые тексты)
- Отсутствие плана на случай сбоя модели или API (нет резервного сценария)
- Передача в промпт чувствительных данных без понимания политики хранения данных у провайдера
- Ожидание, что один и тот же промпт одинаково хорошо сработает на всех моделях
- Автоматизация задачи целиком вместо постепенного внедрения с контролем на старте

9. Практический чек-лист перед запуском

Начинающий

- ☐ Определена задача, где цена ошибки приемлема на старте
- ☐ Выбран подходящий тип инструмента (чат / API / агент / RAG)
- ☐ Есть способ проверить качество результата (человеком или тестами)
- ☐ Настроены лимиты и логирование, если используется API или агент
- ☐ Понятно, какие данные передаются модели и на каких условиях